



Lab 3: 代码评审与单元测试





代码评审



实验要求

- 针对Lab1所完成的代码，进行代码评审(走查)，从代码规范性和正确性角度对代码进行评价；
- 使用以下两个工具完成实验：
 - Checkstyle
 - SpotBugs

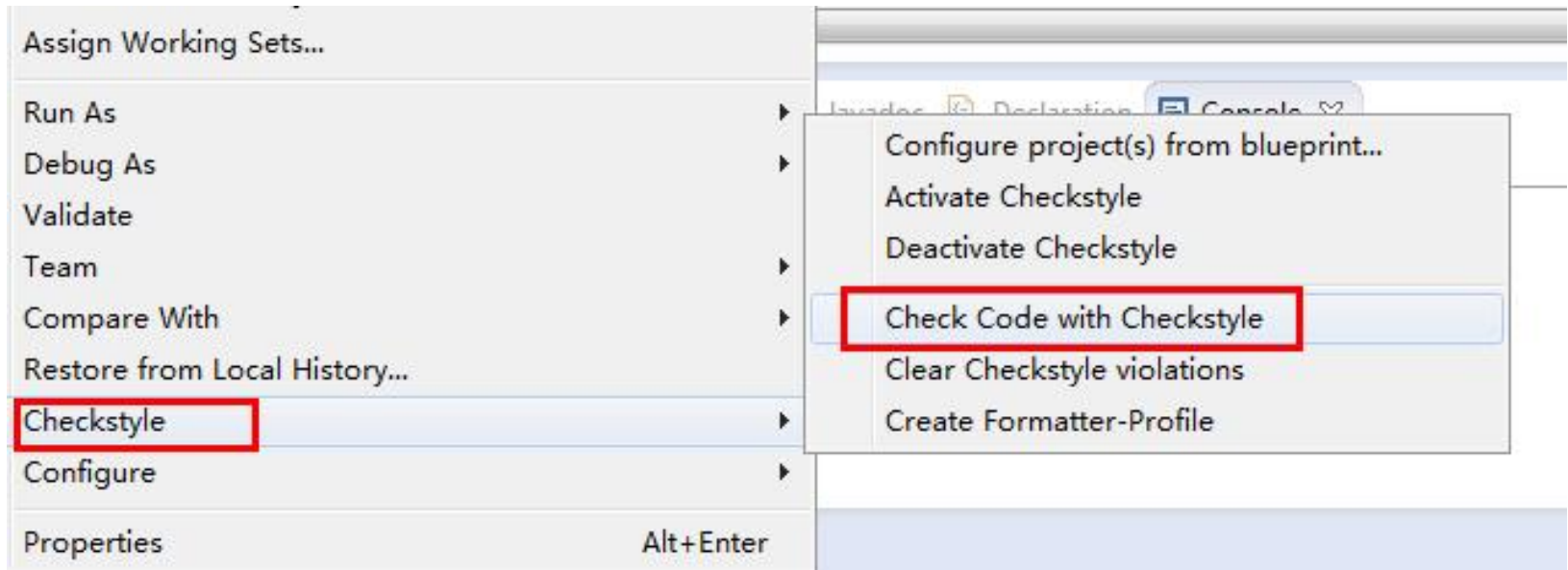
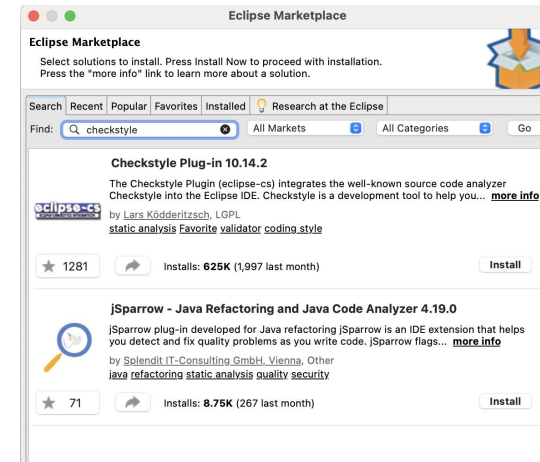
Checkstyle

- CheckStyle帮助Java开发人员遵守某些编码规范，能自动化代码规范检查过程，从而使开发人员从这项重要但是枯燥的任务中解脱出来。请从官网上阅读详细使用手册：<https://checkstyle.org/index.html>
- CheckStyle检验的主要内容：
 - Javadoc注释
 - 命名约定
 - 标题
 - Import语句
 - 体积大小
 - 空白
 - 修饰符
 - 块
 - 代码问题
 - 类设计

Checkstyle in Eclipse

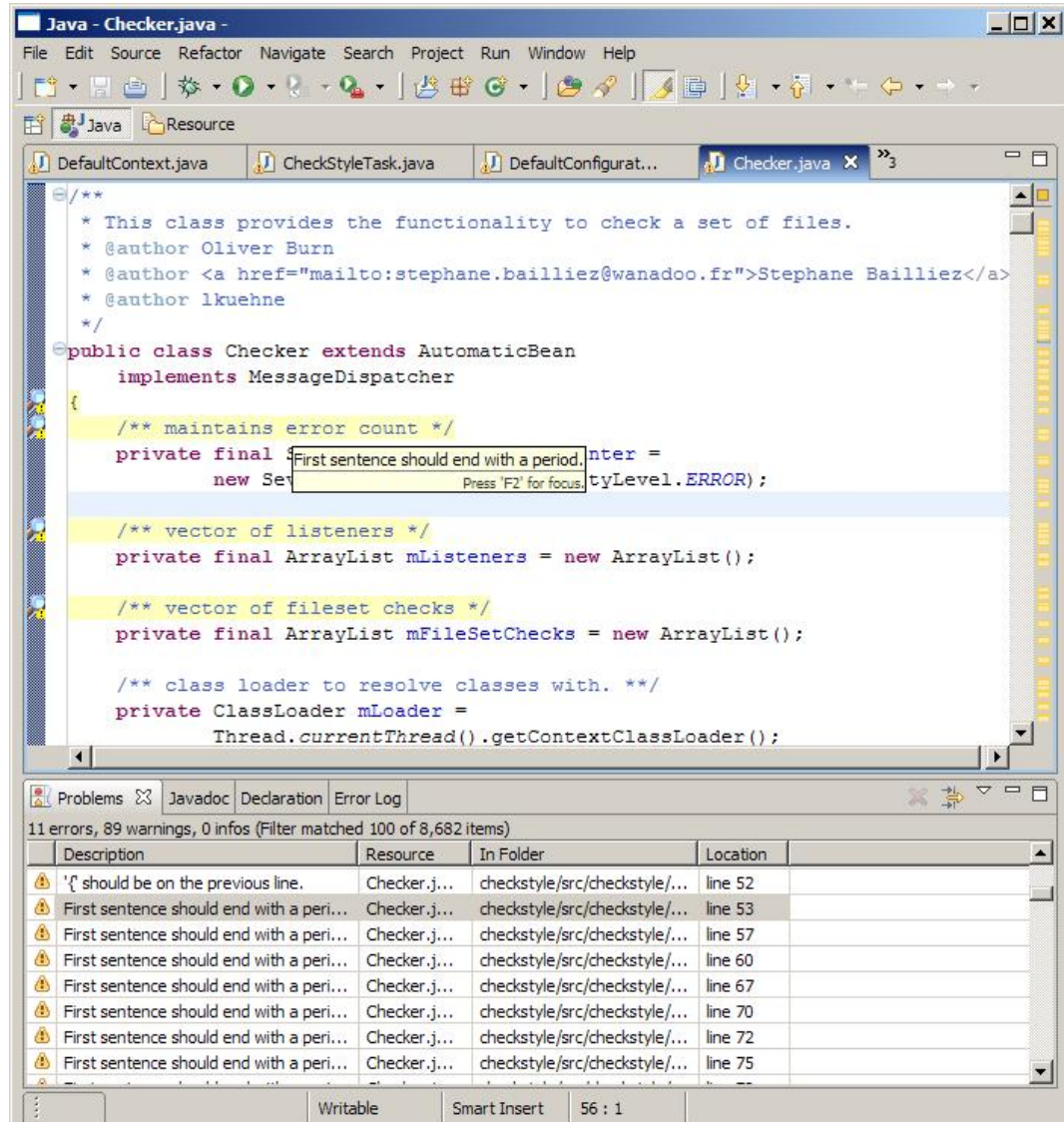
■ Eclipse下的安装和使用

- 从Eclipse Marketplace中搜索CheckStyle安装
- 安装成功后，选中工程，右键选择Checkstyle->check code with checkstyle，检查错误即可。

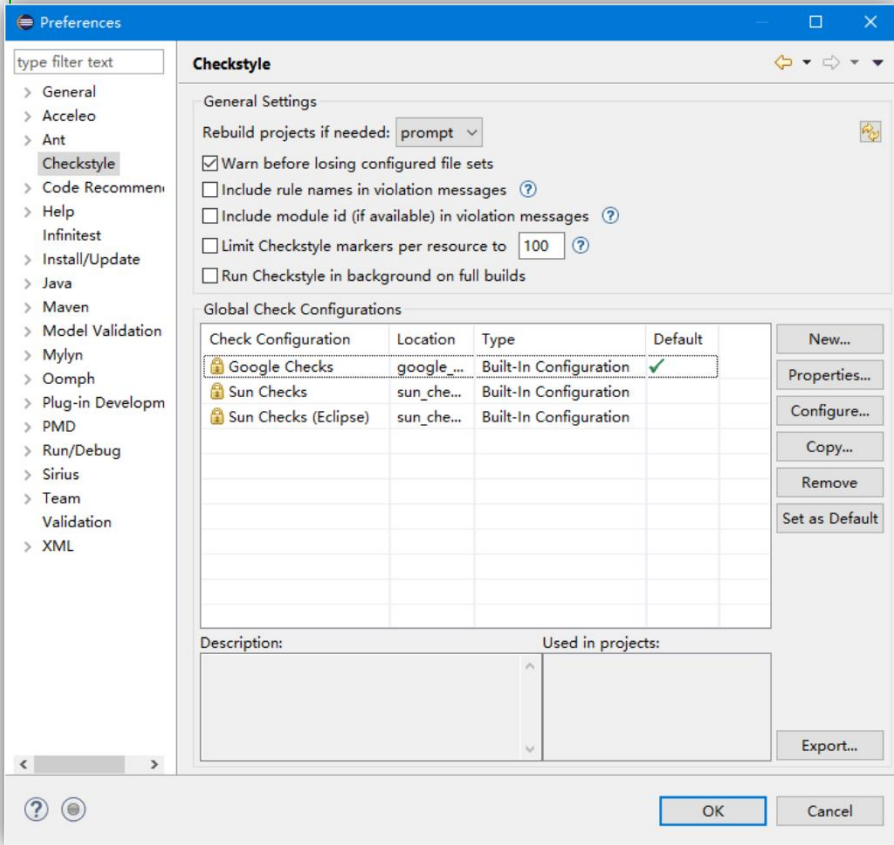


Checkstyle in Eclipse

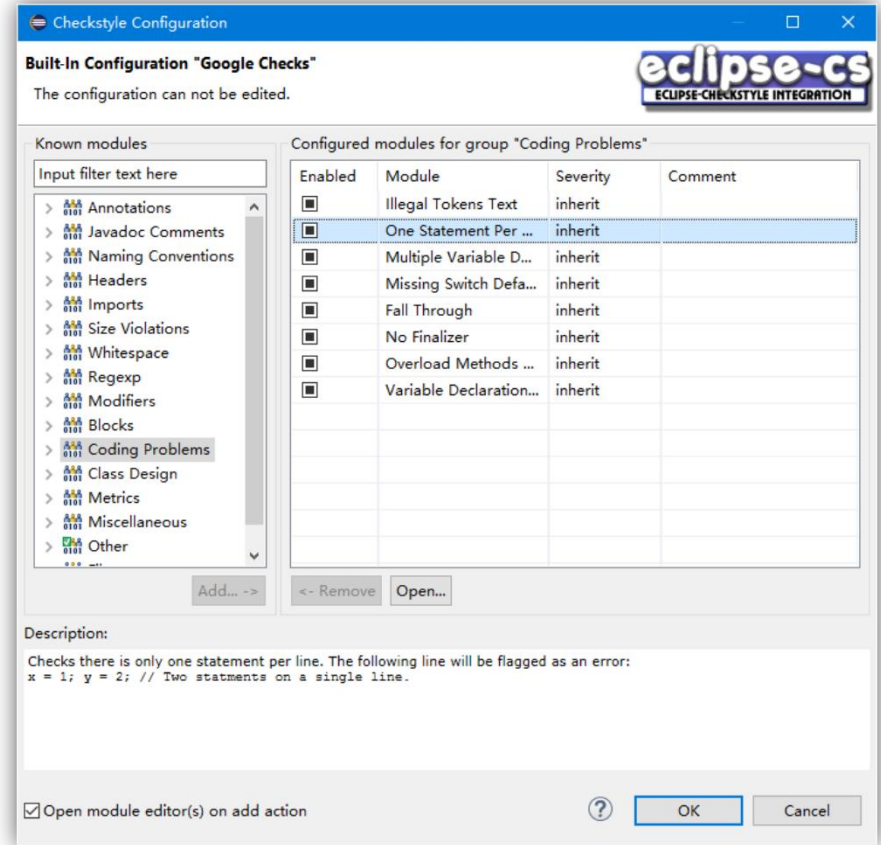
- 检查结果在每行开始显示图标，鼠标移动至其上时显示详细问题；



Checkstyle in Eclipse



此处选择编码规范

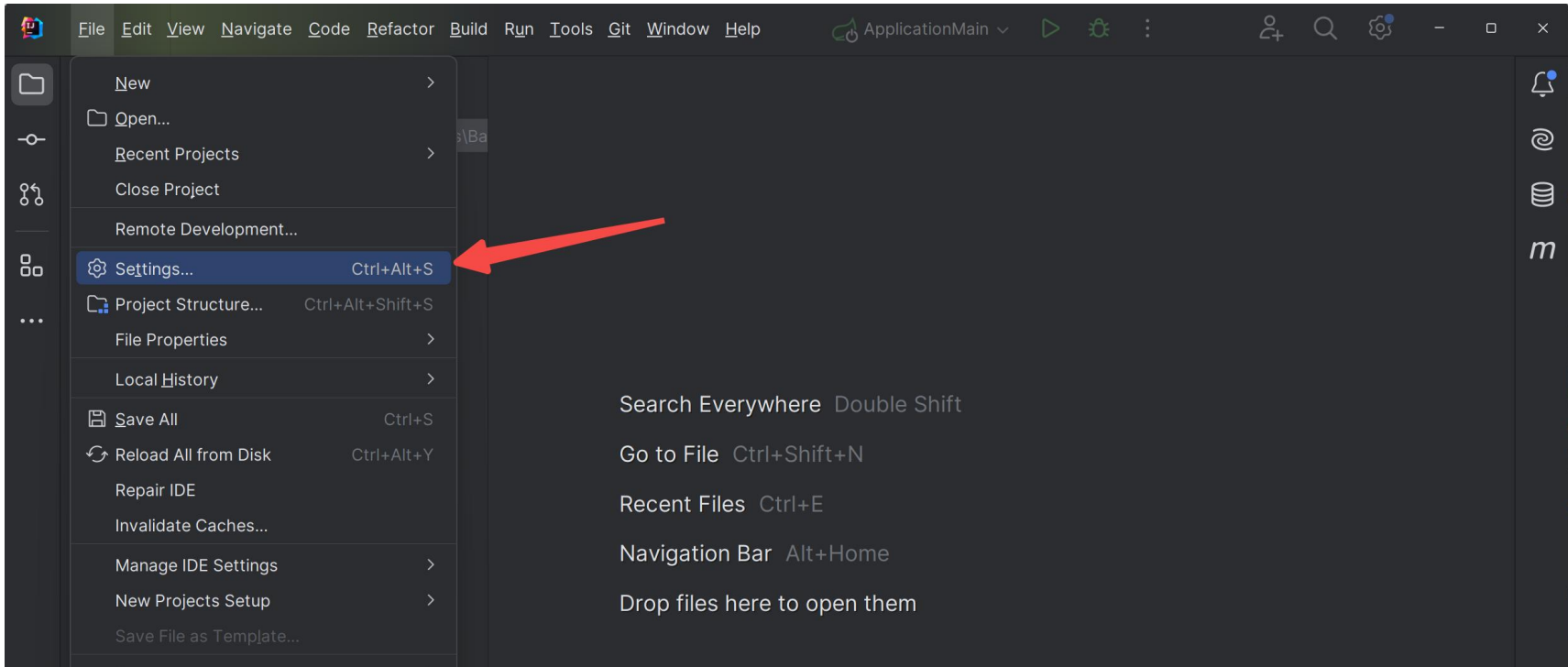


此处阅读规范详情

Checkstyle in IDEA

■ IDEA下的安装

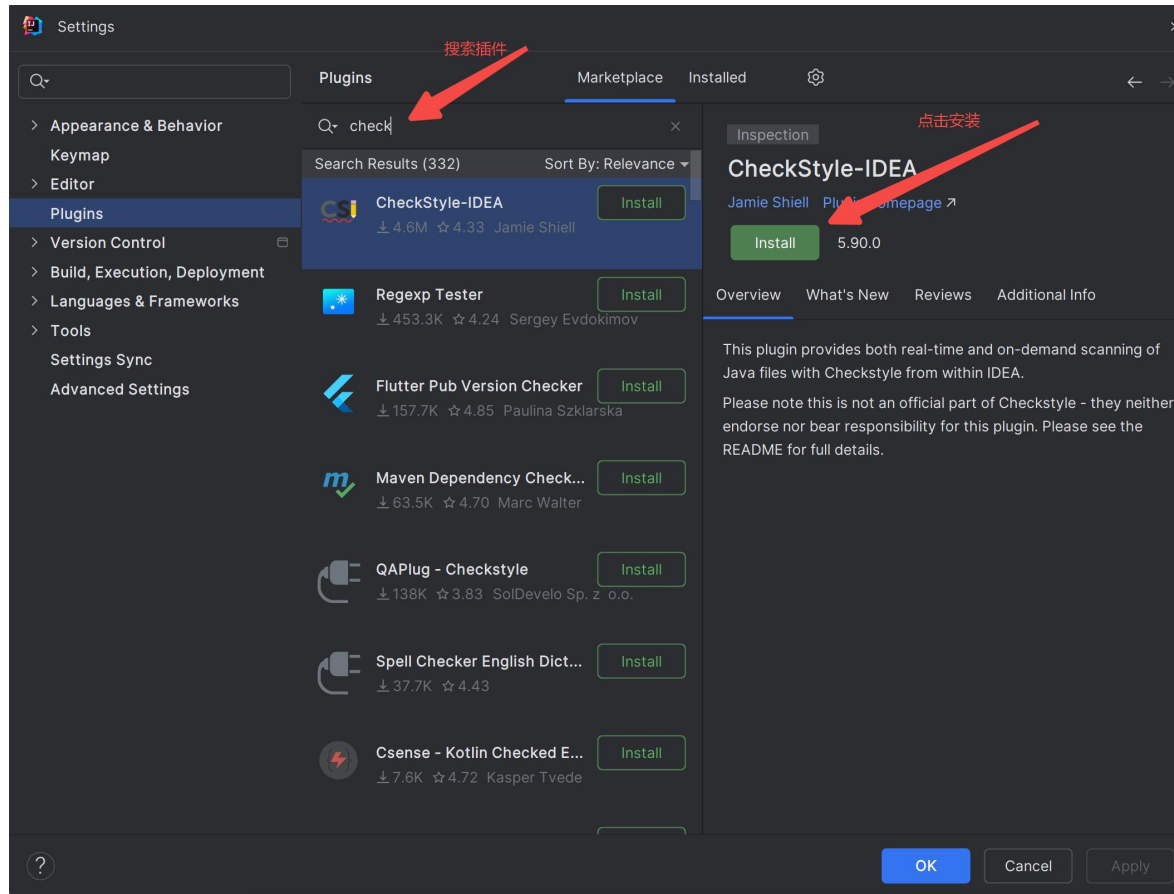
- 打开IDEA菜单栏Settings中的plugins，搜索CheckStyle-IDEA并安装



Checkstyle in IDEA

■ IDEA下的安装

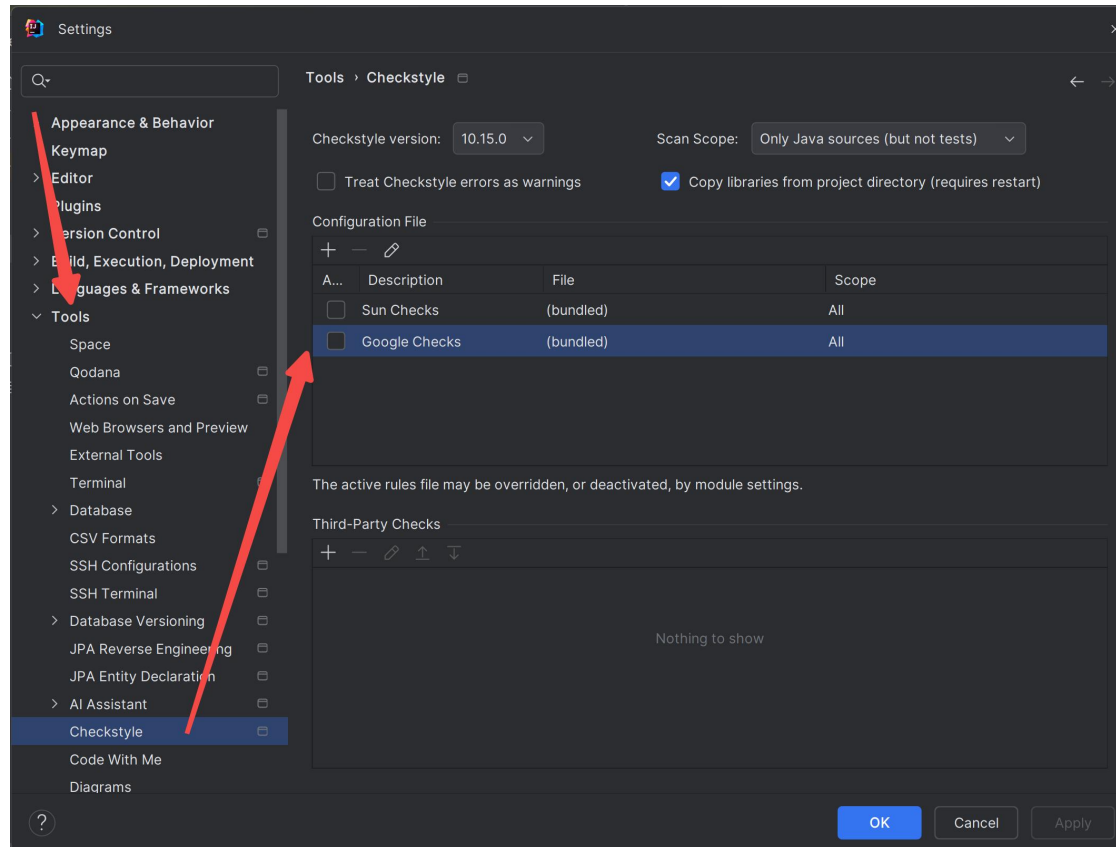
- 打开IDEA菜单栏Settings中的plugins，搜索CheckStyle-IDEA并安装



Checkstyle in IDEA

■ 配置CheckStyle

- 按如下步骤点击，可以选择配置内置的配置文件，以Google Checks为例，选中后Apply



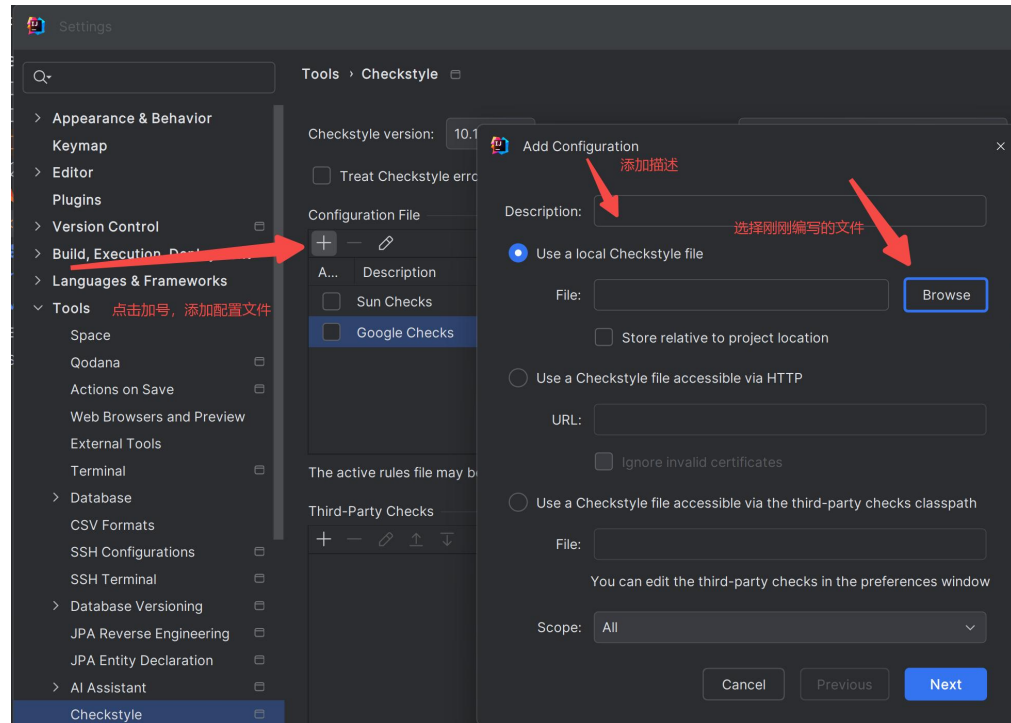
Checkstyle in IDEA

■ 配置CheckStyle

- 也可以自己编写配置文件并导入，在本地新建checkstyle.xml文件，里面的内容示例如下：

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE module PUBLIC "-//Puppy Crawl//DTD Check Configuration 1.3//EN"
    "http://www.puppcrawl.com/dtds/configuration_1_3.dtd">

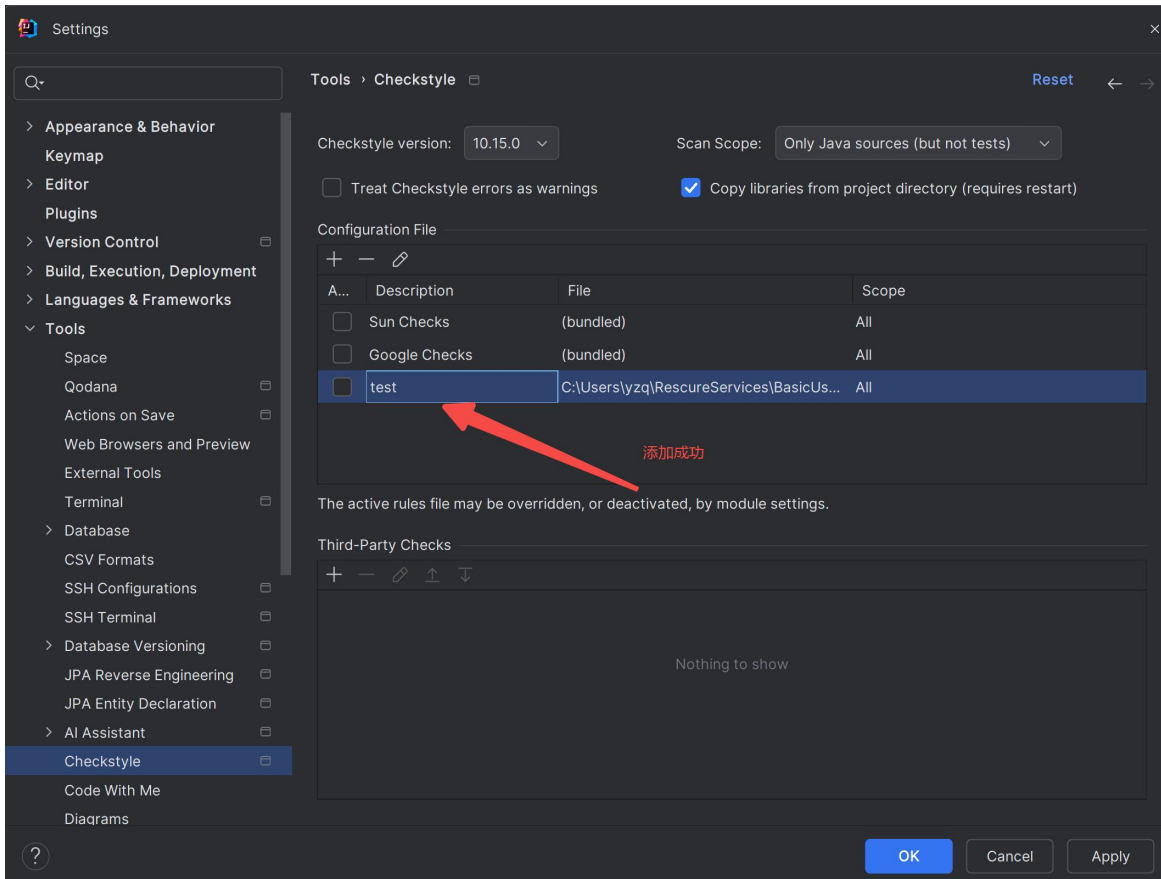
<module name="Checker">
    <!-- 官方主页: https://checkstyle.sourceforge.io -->
    <!-- 检查文件是否以一个空行结束 -->
    <module name="NewlineAtEndOfFile"/>
    <!-- 文件长度不超过 1500 行 -->
    <module name="FileLength">
        <property name="max" value="1500"/>
    </module>
    <!-- 每行不超过 120 个字 -->
    <module name="LineLength">
        <property name="max" value="150"/>
    </module>
    <module name="TreeWalker">
        <!-- import 中避免星号 "*" -->
        <module name="AvoidStarImport"/>
        <!-- 检查是否从非法的包中导入了类 -->
        <module name="IllegalImport"/>
        <!-- 检查是否导入了多余的包 -->
        <module name="RedundantImport"/>
    </module>
</module>
```



Checkstyle in IDEA

■ 配置CheckStyle

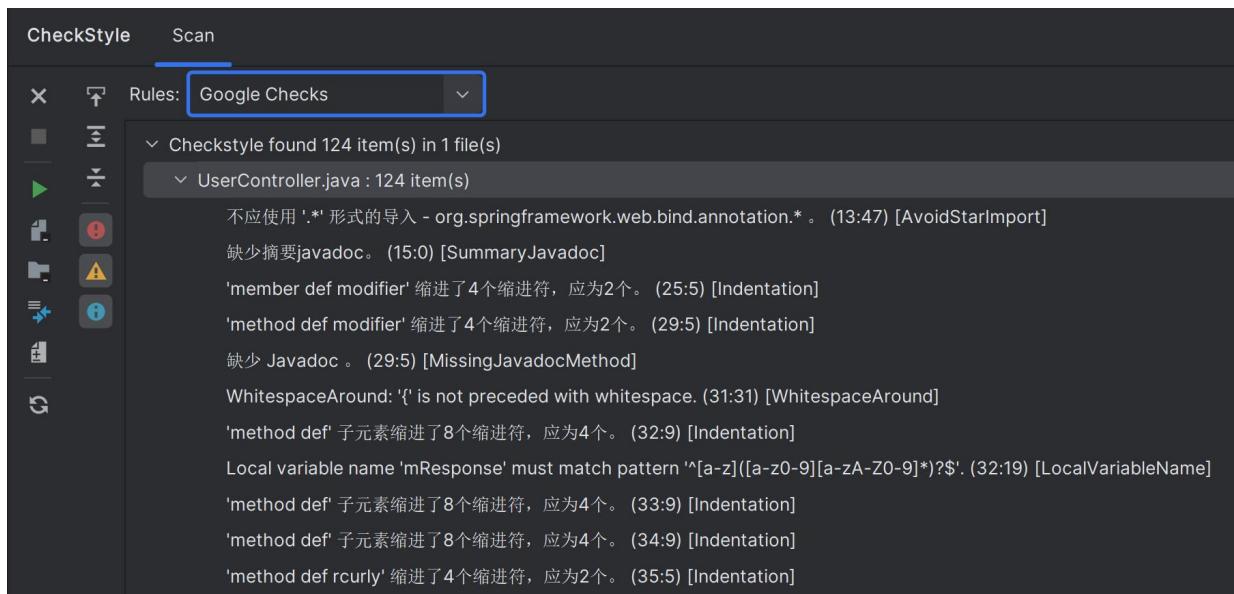
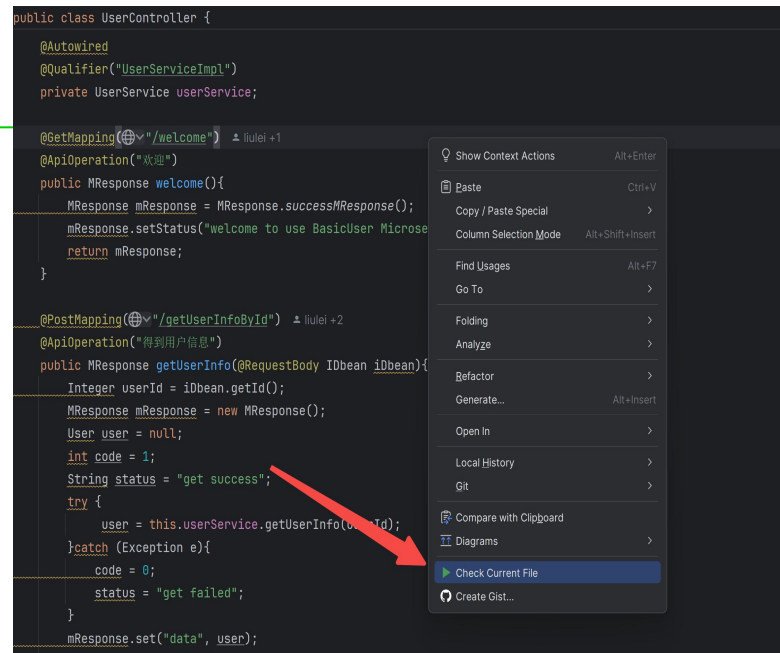
- 勾选指定的规范使之生效。



Checkstyle in IDEA

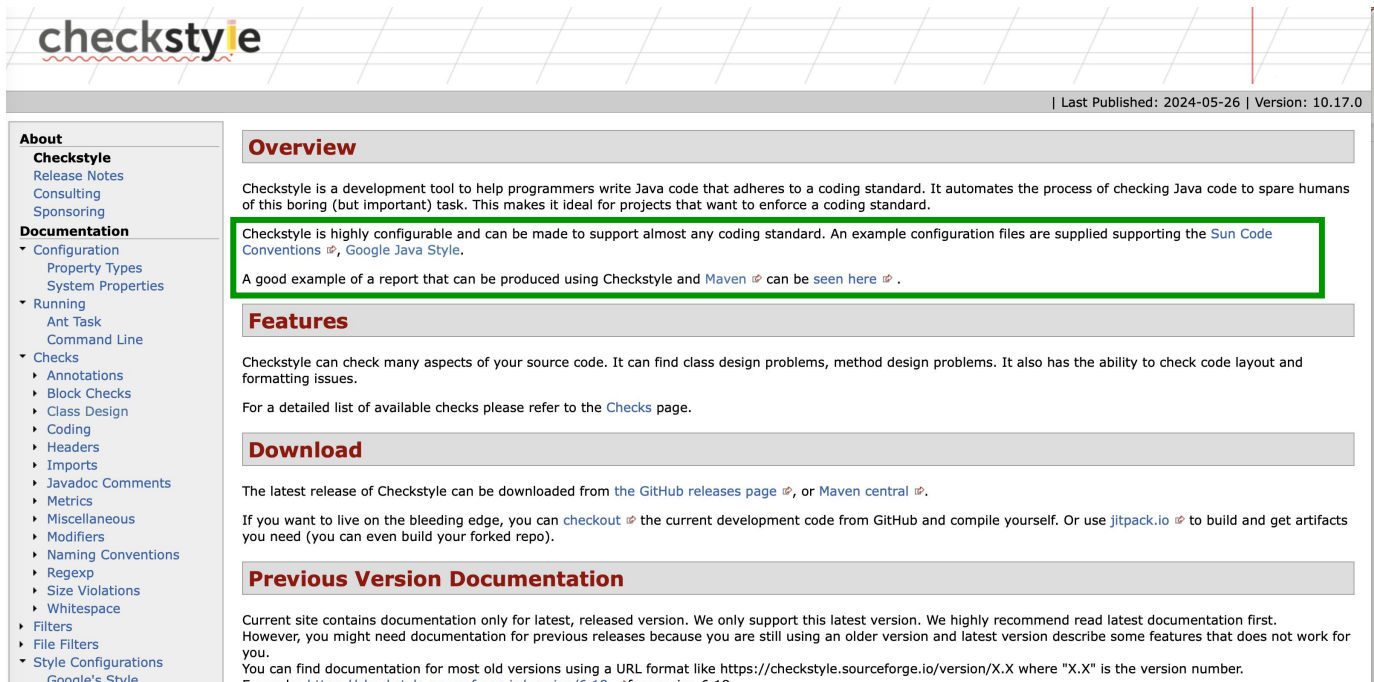
■ 使用

- 打开某一代码文件，右键点击 Check Current File 勾选指定的规范使之生效。



CheckStyle的帮助

- 从<https://checkstyle.org/index.html>学习：
 - 如何安装和配置
 - 如何自定义配置
 - 每个问题的具体含义以及如何修正
- 由此理解Java编码规范。



SpotBugs

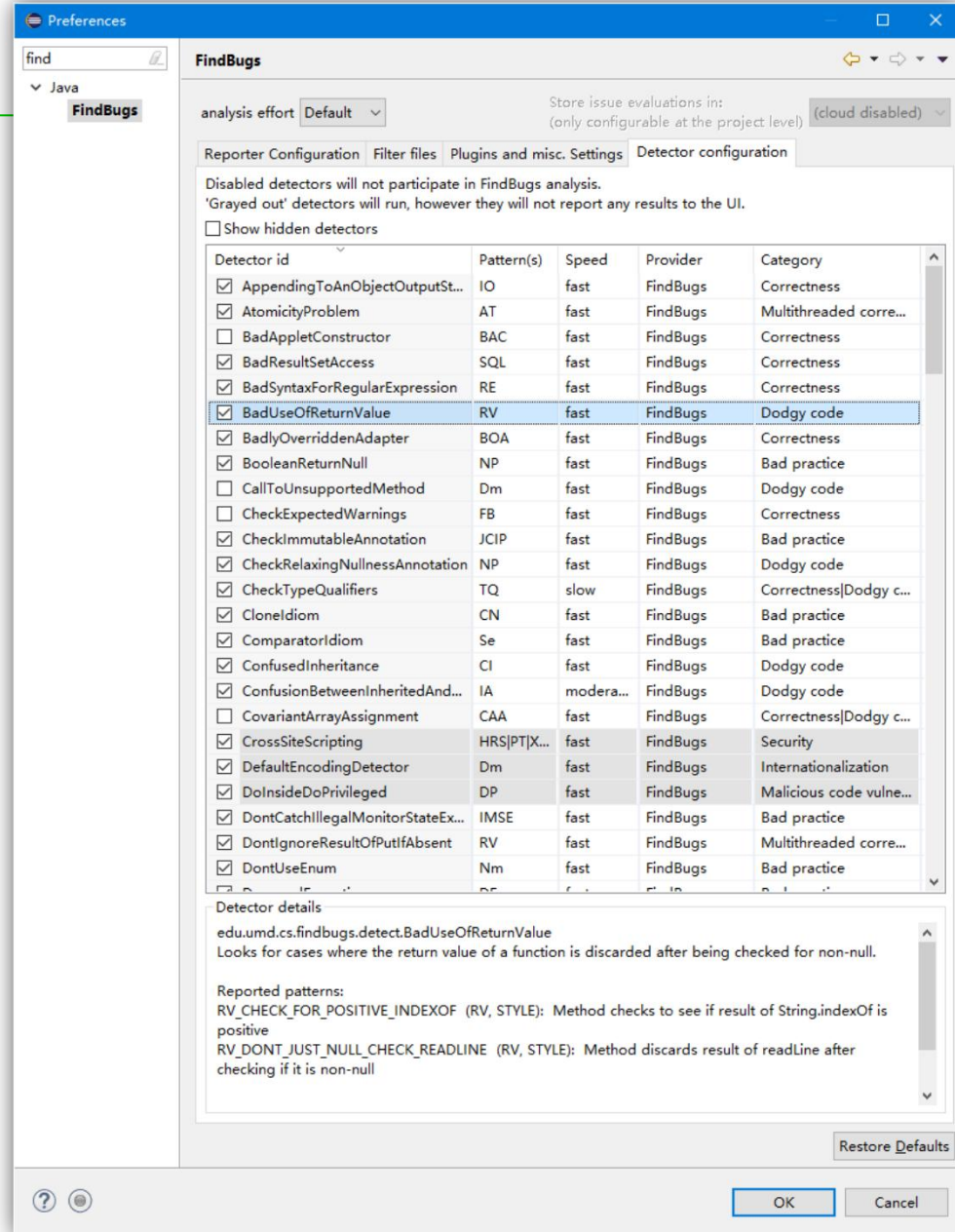
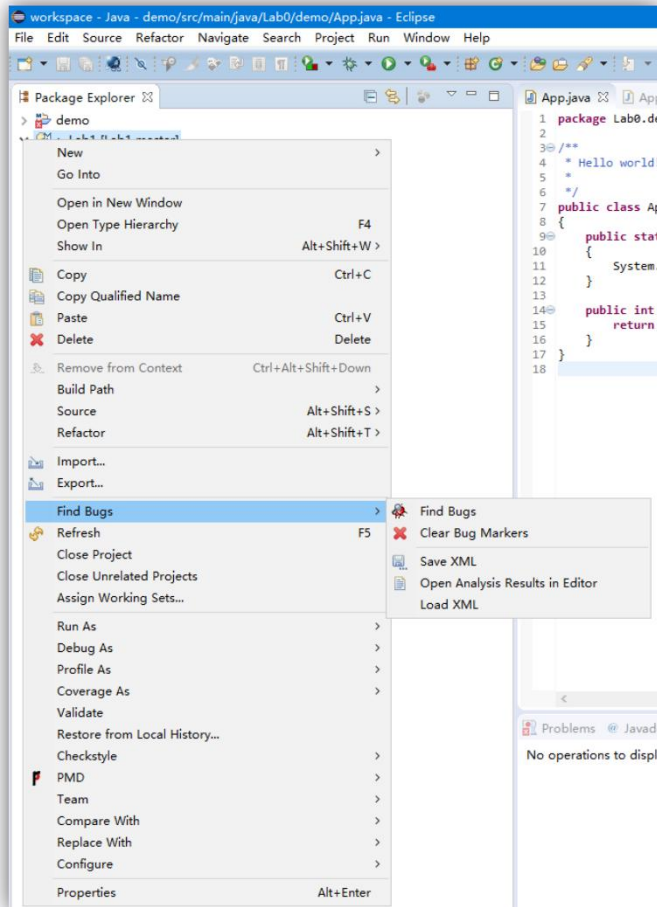
- **SpotBugs**: Java静态代码分析工具，不注重样式或者格式，专注于寻找真正的缺陷或者潜在的性能问题，帮助开发者提高代码质量以及排除隐含的缺陷，可以在不实际运行程序的情况对软件进行分析。

<https://spotbugs.github.io>

- 常见的检测类型如下：
 - 正确性(Correctness): 在某种情况下会导致bug，如错误的强制类型转换等。
 - 最佳实践反例(Bad practice): 这种类别下的代码违反了公认的最佳实践标准，比如某个类实现了equals方法但未实现hashCode方法等。
 - 多线程正确性(Multithreaded correctness): 关注于同步和多线程问题。
 - 性能(Performance): 潜在的性能问题。
 - 安全(Security): 安全相关。
 - 高危(Dodgy): 该类型下的问题代码导致bug的可能性很高。

SpotBugs in Eclipse

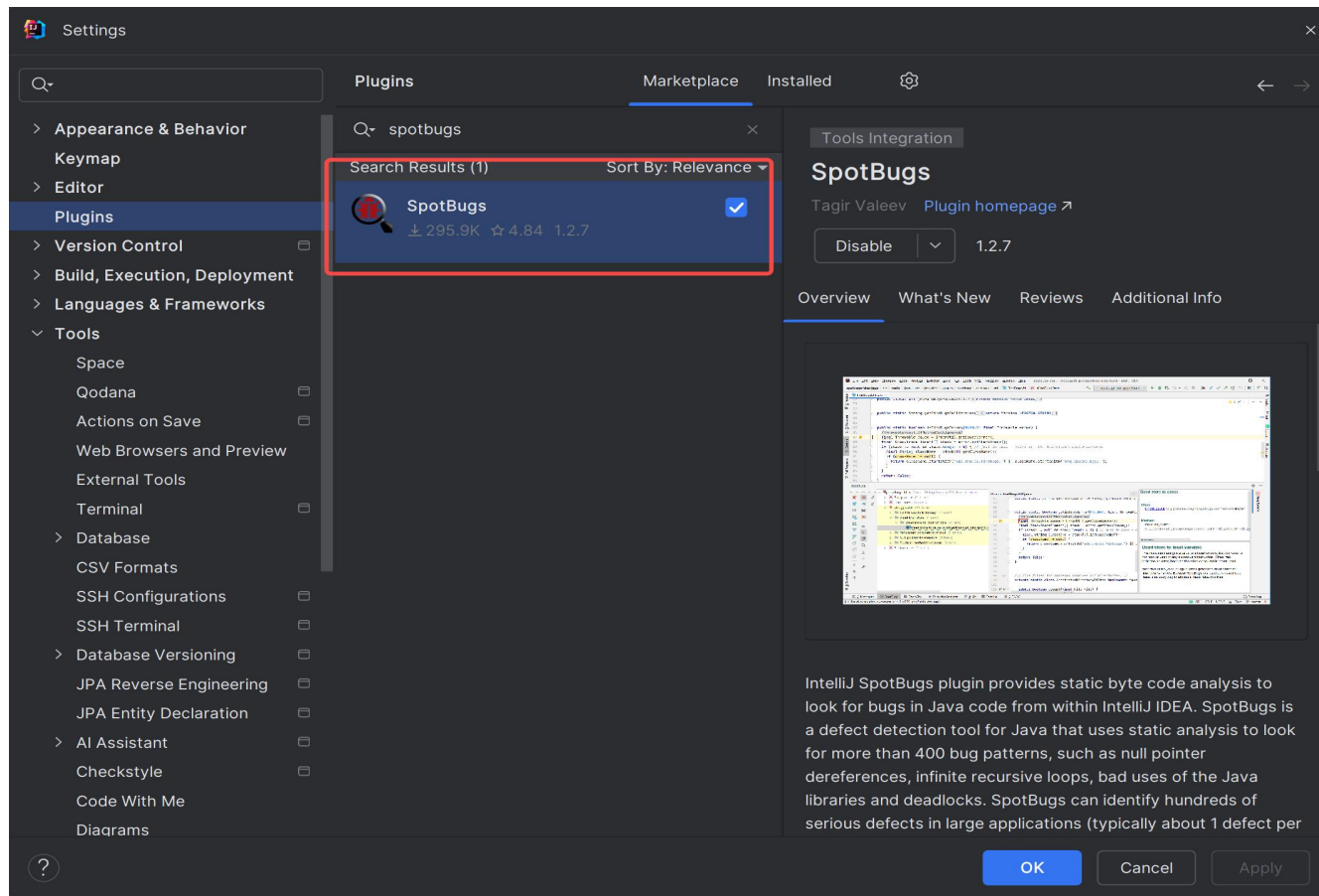
- 通过Eclipse Marketplace安装使用。



SpotBugs in IDEA

■ 安装

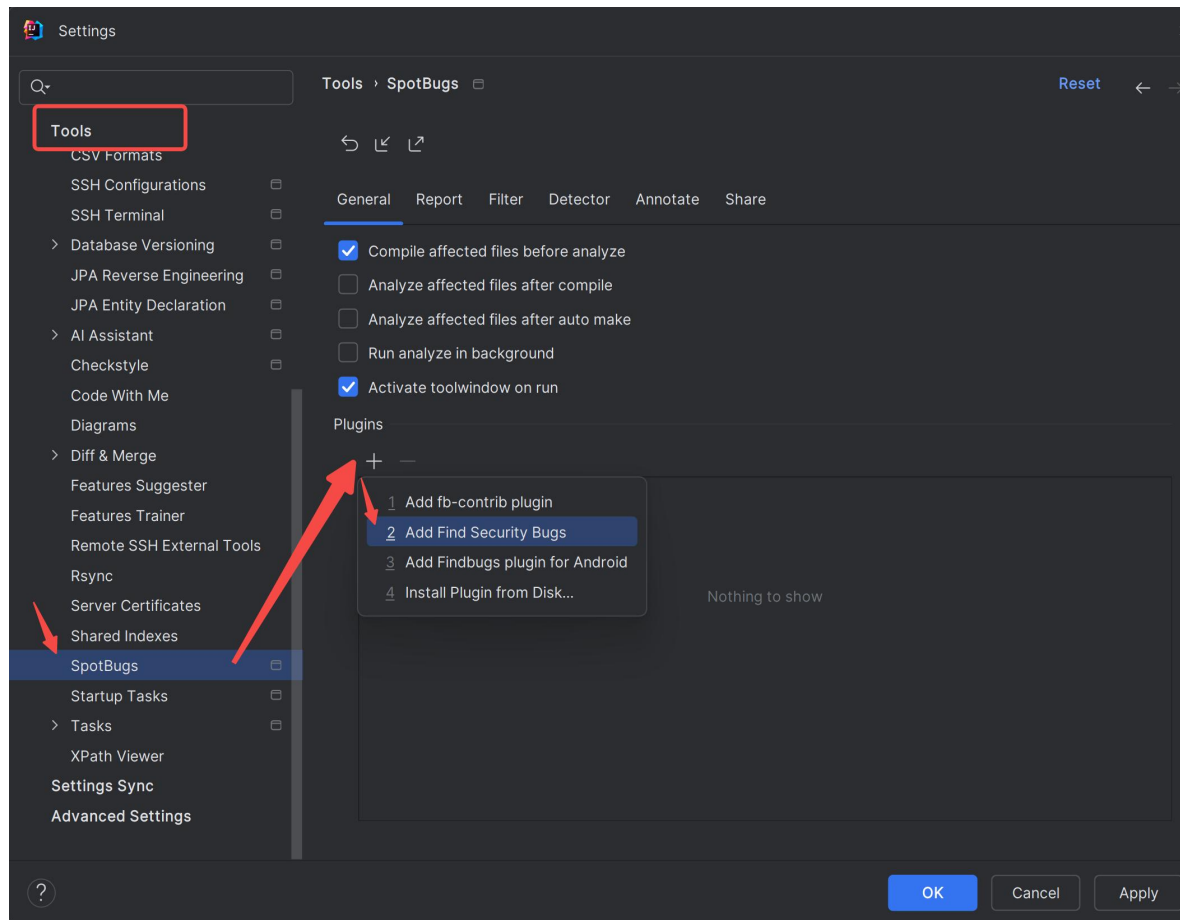
- 在IDEA插件市场搜索SpotBugs，点击Install安装，安装后重启IDEA。



SpotBugs in IDEA

■ 配置安全规则

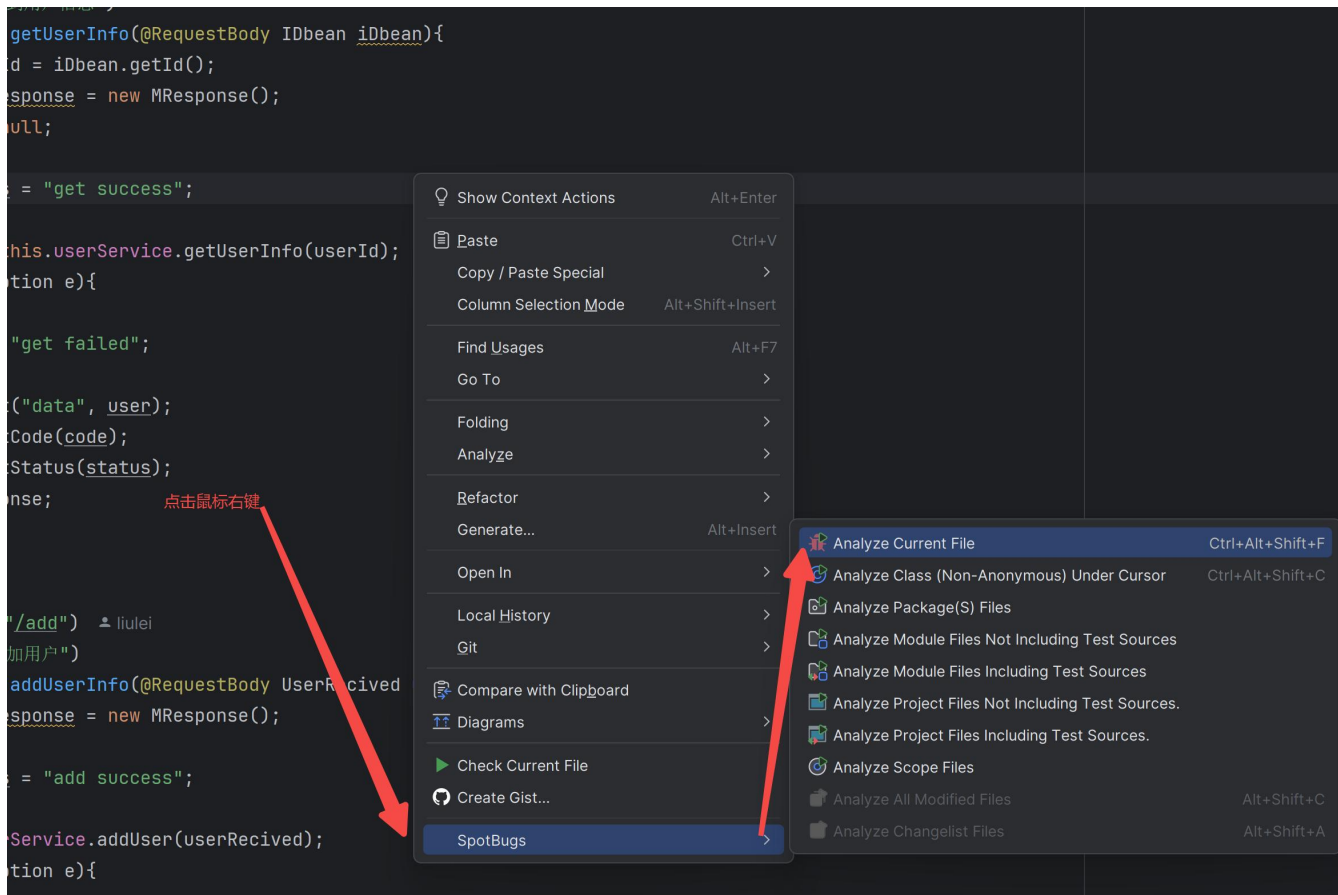
- 在设置的Tools下，点击SpotBugs，按下图所示点击加号，添加规则。



SpotBugs in IDEA

■ 使用

- 在某一代码文件上，右键选择Analyze Current File。



SpotBugs in IDEA

■ 使用

- 在某一代码文件上，右键选择Analyze Current File。
- 若代码中包含潜在BUG，会在控制台输出



SpotBugs in IDEA

- 阅读网站 <https://spotbugs.github.io> 了解详细信息
- 阅读Bug description学习常见Bug和相关知识



SpotBugs

Find bugs in Java Programs

Check it out on GitHub

SpotBugs is a program which uses static analysis to look for bugs in Java code. It is free software, distributed under the terms of the [GNU Lesser General Public License](#).

SpotBugs is a fork of [FindBugs](#) (which is now an abandoned project), carrying on from the point where it left off with support of its community. Please check [the official manual](#) for details.

SpotBugs requires JRE (or JDK) 1.8.0 or later to run. However, it can analyze programs compiled for any version of Java.

Bug Descriptions

SpotBugs checks for more than 400 bug patterns. Bug descriptions can be found [here](#).

Descriptions are also available in [Japanese](#).

Using SpotBugs

SpotBugs can be used standalone and through several integrations, including:

- [Ant](#)
- [Maven](#)
- [Gradle](#)
- [Eclipse](#)

To automatically configure the Eclipse SpotBugs plugin from the SpotBugs Maven plugin configuration, use [m2e-code-quality](#).

Extensions

SpotBugs is extensible. New detectors can be added through plugins. Popular SpotBugs plugins include:

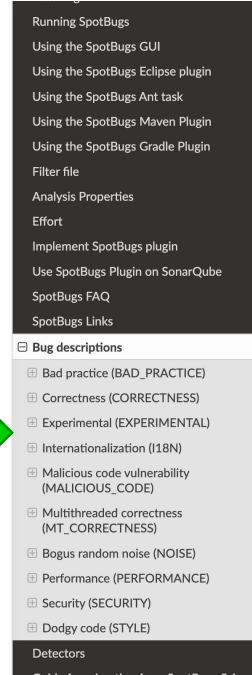
[Bug Descriptions](#)

[Using SpotBugs](#)

[Extensions](#)

[License of SpotBugs Logo](#)

[Support or Contact](#)



Running SpotBugs

- Using the SpotBugs GUI
- Using the SpotBugs Eclipse plugin
- Using the SpotBugs Ant task
- Using the SpotBugs Maven Plugin
- Using the SpotBugs Gradle Plugin

Filter file

Analysis Properties

Effort

Implement SpotBugs plugin

Use SpotBugs Plugin on SonarQube

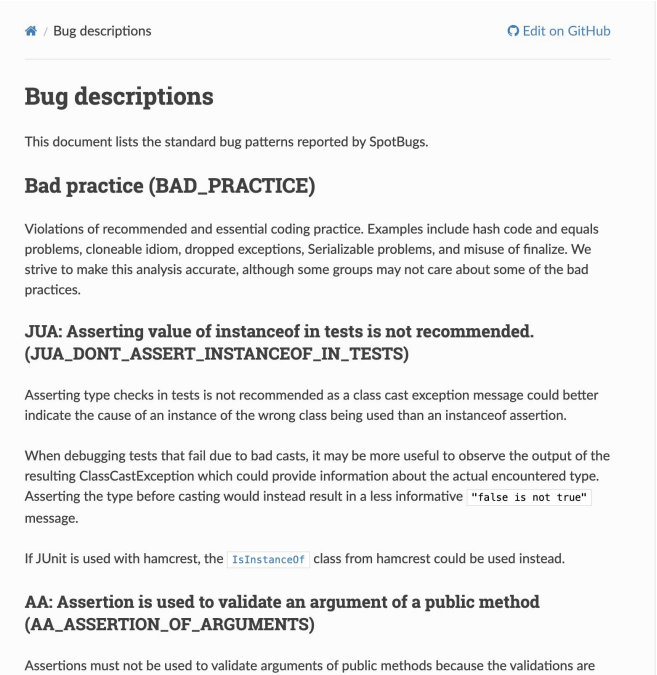
SpotBugs FAQ

SpotBugs Links

Bug descriptions

- Bad practice (BAD_PRACTICE)
- Correctness (CORRECTNESS)
- Experimental (EXPERIMENTAL)
- Internationalization (I18N)
- Malicious code vulnerability (MALICIOUS_CODE)
- Multithreaded correctness (MT_CORRECTNESS)
- Bogus random noise (NOISE)
- Performance (PERFORMANCE)
- Security (SECURITY)
- Dodgy code (STYLE)

Detectors



🏠 / Bug descriptions

[Edit on GitHub](#)

Bug descriptions

This document lists the standard bug patterns reported by SpotBugs.

Bad practice (BAD_PRACTICE)

Violations of recommended and essential coding practice. Examples include hash code and equals problems, cloneable idiom, dropped exceptions, Serializable problems, and misuse of finalize. We strive to make this analysis accurate, although some groups may not care about some of the bad practices.

JUA: Asserting value of instanceof in tests is not recommended. (JUA_DONT_ASSERT_INSTANCEOF_IN_TESTS)

Asserting type checks in tests is not recommended as a class cast exception message could better indicate the cause of an instance of the wrong class being used than an instanceof assertion.

When debugging tests that fail due to bad casts, it may be more useful to observe the output of the resulting ClassCastException which could provide information about the actual encountered type. Asserting the type before casting would instead result in a less informative "false is not true" message.

If JUnit is used with hamcrest, the `IsInstanceOf` class from hamcrest could be used instead.

AA: Assertion is used to validate an argument of a public method (AA_ASSERTION_OF_ARGUMENTS)

Assertions must not be used to validate arguments of public methods because the validations are

代码评审部分：评判标准

- 是否可在IDE中顺利配置工具并启动执行；
- 能够充分理解CheckStyles、SpotBugs工具所检测的不符合规范的常见代码问题；
- 根据工具报告的问题，对代码做了充分的改进；
- 实验报告的规范性、完整性。



单元测试



实验目标

- 针对Lab1完成的程序，设计黑盒测试用例和白盒测试用例。
- 在JUnit环境下撰写测试代码并执行测试。
- 使用EclEmma或IDE自带工具统计测试的覆盖度。

Step 1: 设计黑盒测试用例

- 利用黑盒测试的等价类和边界值方法，为Lab1待测程序设计一组测试用例。
- 测试对象(五选一):
 - `String queryBridgeWords(String word1, String word2)`: 查询桥接词
 - `String generateNewText(String inputText)`: 根据bridge word生成新文本
 - `String calcShortestPath(String word1, String word2)`: 计算两个单词之间的最短路径
 - `Double calPageRank(String word)`: 计算单词的PR值
 - `String randomWalk()`: 随机游走
- 根据Lab1的要求，给出所选被测函数的需求规约描述；
- 每个测试用例由输入数据和期望输出两部分组成。

Step 2: 使用JUnit编写黑盒测试用例并执行

- 在Lab1的Git仓库里，建立新的Git分支，命名为Lab3b (b代表black-box testing);
- 针对每个测试用例撰写testcase;
- 执行测试用例：
 - 执行，产生结果，记录实际输出;
 - 记录、分析结果;
 - 针对失败的测试用例，发现代码的问题，并修改代码;
- 重复上一步，直到所有的测试用例都完全通过为止。
- 将Lab3b合并到master分支，并推送至GitHub。

Step 3: 设计白盒测试用例

- 针对Lab3b分支的当前代码，对以下函数(三选一)使用基本路径法设计白盒测试用例：
 - `String queryBridgeWords(type G, String word1, String word2)`
查询桥接词
 - `String calcShortestPath(type G, String word1, String word2):`
计算两个单词之间的最短路径
 - `String randomWalk(type G):` 随机游走
- 每个测试用例由输入数据和期望输出两部分组成。

Step 4: 使用JUnit编写并执行白盒测试代码

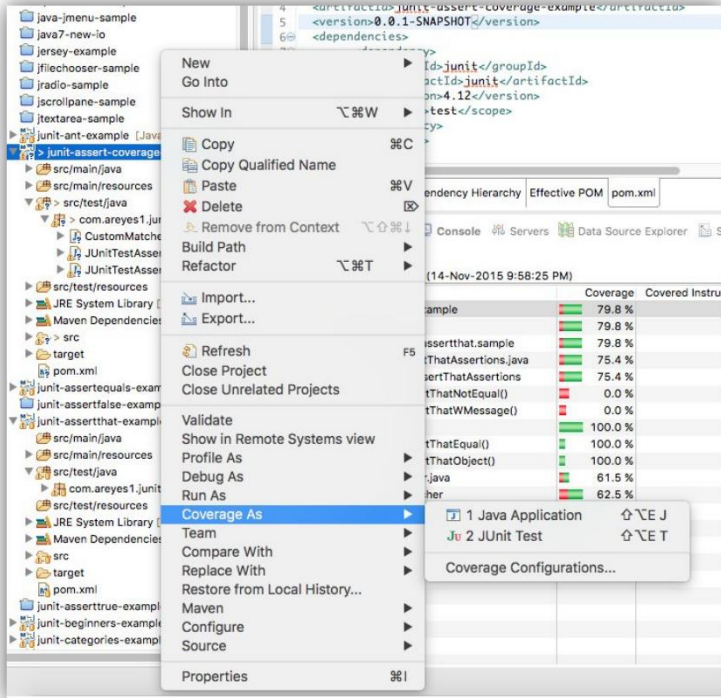
- 在Lab1的Git仓库里，建立新的Git分支，命名为Lab3w (w代表white-box testing);
- 针对每个测试用例撰写testcase;
- 执行测试用例：
 - 执行，产生结果，记录实际输出;
 - 记录、分析结果;
 - 针对失败的测试用例，发现代码的问题，并修改代码;
- 重复上一步，直到所有的测试用例都完全通过为止。
- 将Lab3w合并到master分支，并推送至GitHub。

Step 5: 统计测试覆盖度

- 测试的覆盖率是测试质量的一个重要指标。
 - 可使用Eclemma工具 <https://www.eclemma.org/>
 - 或Eclipse自带Coverage分析工具
 - 或IDEA自带Coverage分析工具
- 在测试过程中，当运行测试程序，覆盖度分析工具可自动分析出被测程序的各行代码被覆盖的情况；
- 代码被覆盖得越全面，测试质量就越好。
- 从工具导出覆盖度分析报告，观察语句覆盖度。

Step 5: 统计测试覆盖度

■ Eclipse下通过Marketplace安装EclEmma



```
//printing fibonacci series upto number
for(int i=1; i<=number; i++){
    System.out.print(fibonacciRecusion(i) + " ");
}

// Java program for Fibonacci number using recursion.
public static int fibonacciRecusion(int number){
    if(number == 1 || number == 2){
        return 1;
    }

    return fibonacciRecusion(number-1) + fibonacciRecusion(number -2); //tail recursion
}

// Java program for Fibonacci number using Loop.
public static int fibonacciLoop(int number){
    if(number == 1 || number == 2){
        return 1;
    }
    int fibo1=1, fibo2=1, fibonacci=1;
    for(int i= 3; i<= number; i++){
        fibonacci = fibo1 + fibo2; //Fibonacci number is sum of previous two Fibonacci number
        fibo1 = fibo2;
        fibo2 = fibonacci;
    }
    return fibonacci; //Fibonacci number
}
```

Red Color - we haven't touch main method - so all RED

Green Color - Code coverage is good for these lines

Step 5: 统计测试覆盖度

■ Eclipse下通过Marketplace安装EclEmma

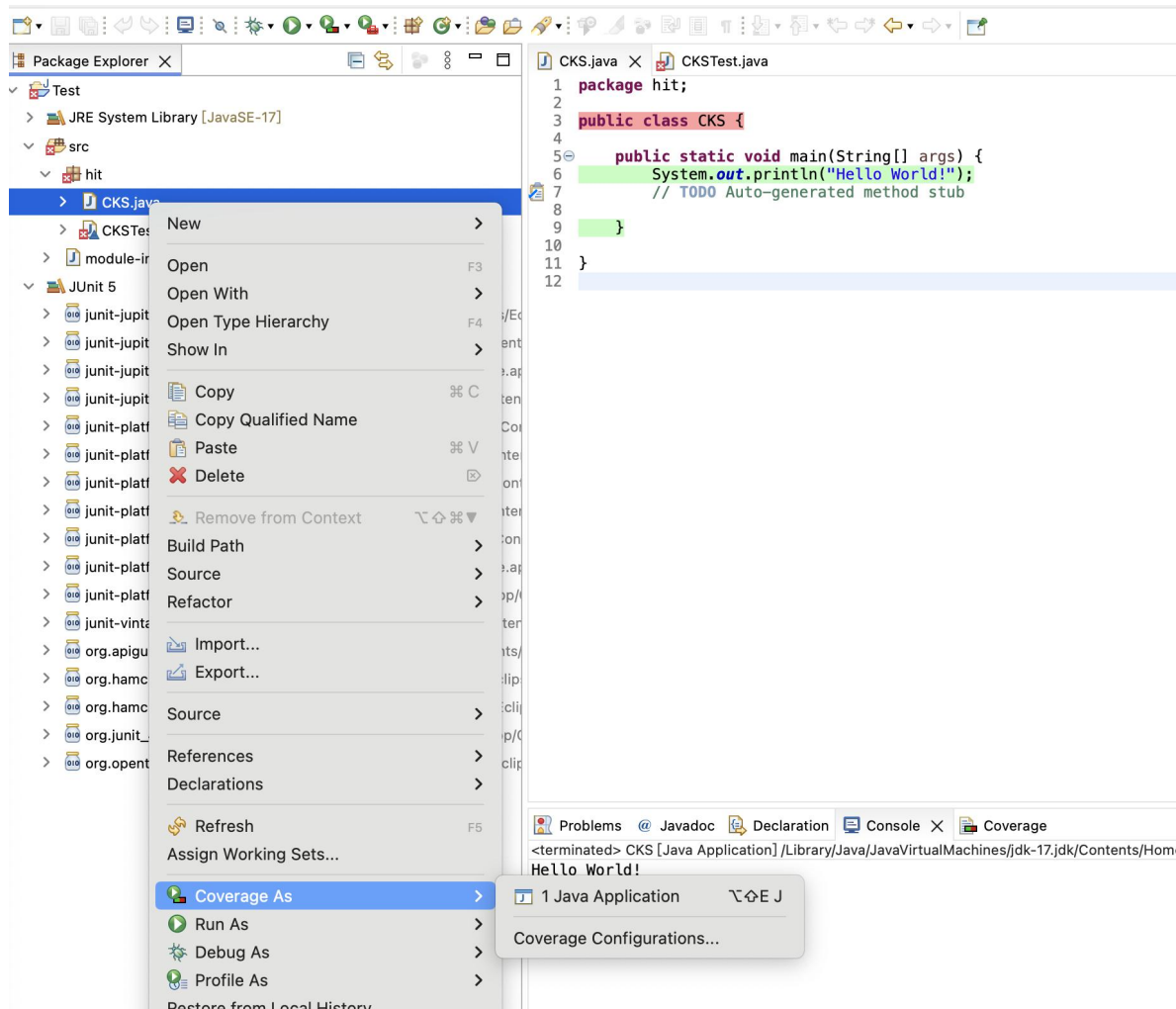
The screenshot displays the Eclipse IDE interface. The top toolbar includes standard development tools. The left sidebar shows the 'JUnit' test runner results, indicating a successful run after 34,898 seconds with 13,009 runs, 0 errors, and 0 failures. Below this, a 'Failures' tab is active, showing a hierarchy of test classes under 'junit.framework.TestSuite'. The main editor window displays the source code of 'CursorableLinkedList.java', with several lines highlighted in green and yellow. The bottom panel is divided into 'Problems', 'Javadoc', 'Declaration', 'Console', and 'Coverage' tabs. The 'Coverage' tab is selected, showing a table of coverage data for the project 'TestAllPackages' as of 31.10.2006 15:04:14.

Element	Coverage	Covered Lines	Total Lines
java - commons-collections	79,5 %	10927	13738
org.apache.commons.collections	74,1 %	3842	5183
ArrayStack.java	86,5 %	32	37
BagUtils.java	86,7 %	13	15
BeanMap.java	72,4 %	155	214
BinaryHeap.java	87,6 %	127	145
BoundedFifoBuffer.java	93,2 %	82	88
BufferOverflowException.java	55,6 %	5	9
BufferUnderflowException.java	88,9 %	8	9
BufferUtils.java	30,8 %	4	13
ClosureUtils.java	93,9 %	31	33
CollectionUtils.java	92,4 %	293	317
ComparatorUtils.java	8,6 %	3	35
CursorableLinkedList.java	85,4 %	444	520

At the bottom of the IDE, the status bar shows 'Writable', 'Smart Insert', and '149 : 28'.

Step 5: 统计测试覆盖度

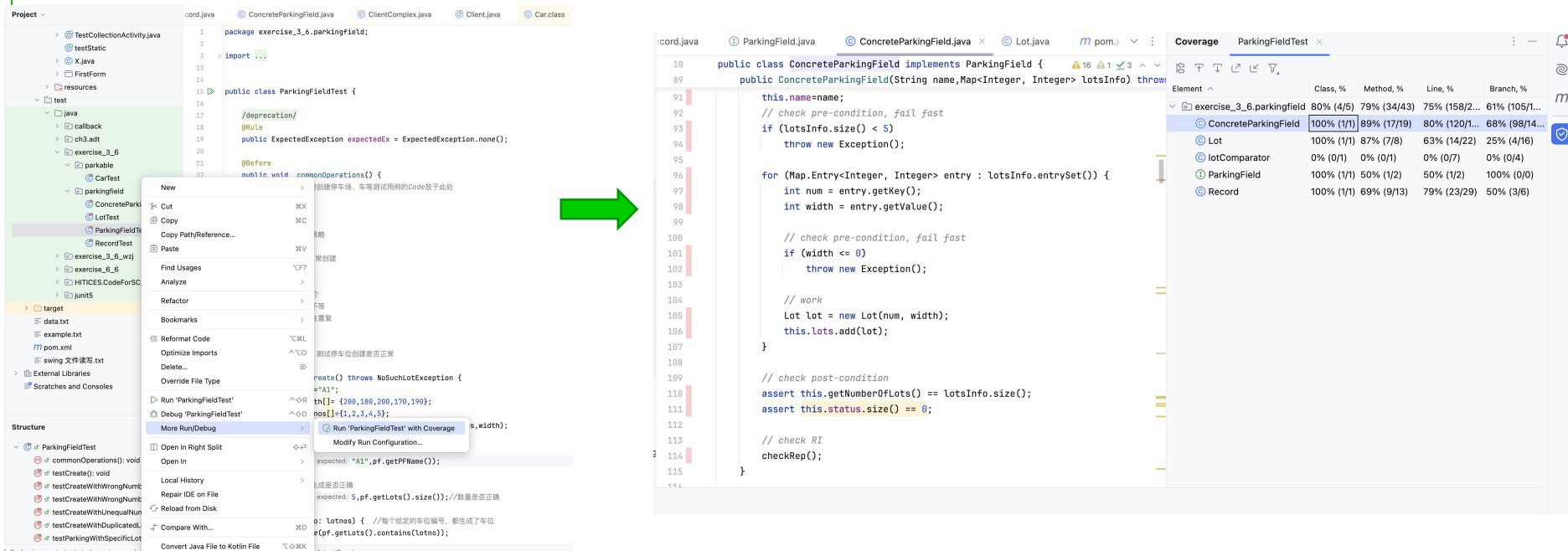
- 采用Eclipse下自带的覆盖度分析工具



Step 5: 统计测试覆盖度

■ IDEA下自带覆盖度分析工具

— <https://www.jetbrains.com/help/idea/code-coverage.html>



The screenshot illustrates the process of running a test with coverage in IntelliJ IDEA. A green arrow points from the 'Run' button in the IDE to the 'Coverage' tool window.

Code Snippet (ConcreteParkingField.java):

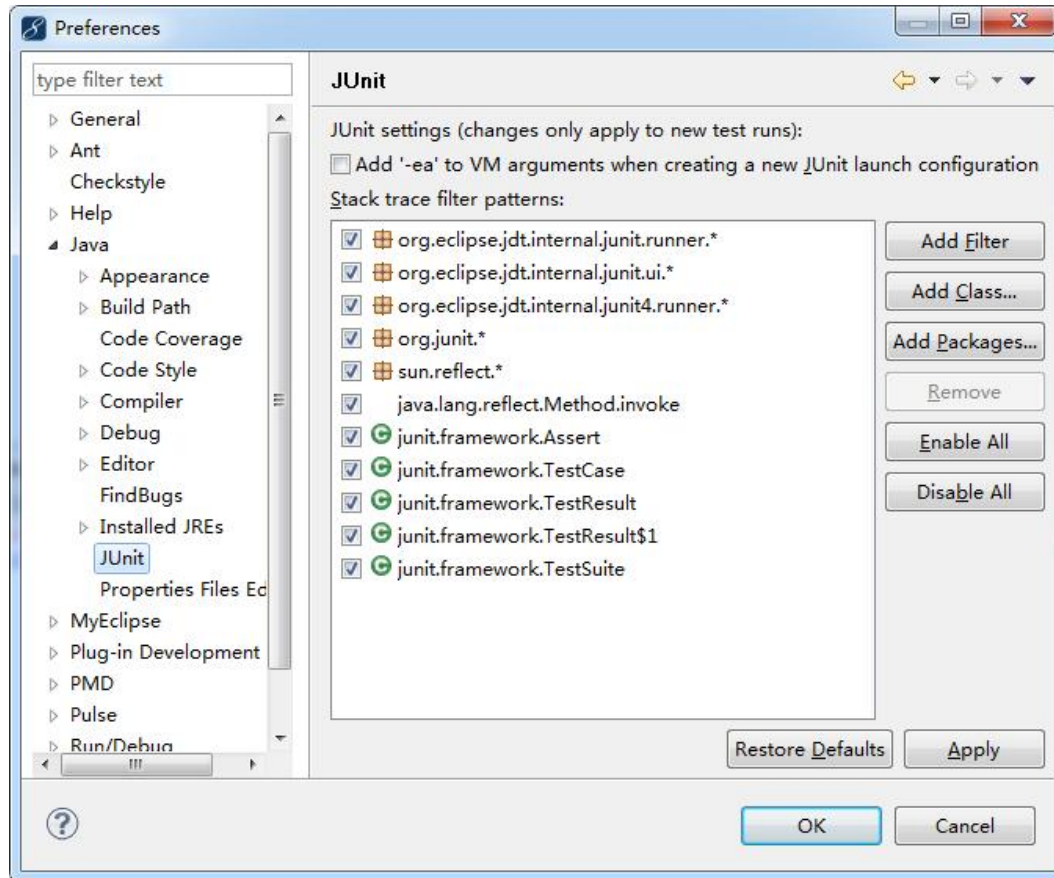
```
public class ConcreteParkingField implements ParkingField {  
    public ConcreteParkingField(String name, Map<Integer, Integer> lotsInfo) throws  
    {  
        this.name=name;  
        // check pre-condition, fail fast  
        if (lotsInfo.size() < 5)  
            throw new Exception();  
        for (Map.Entry<Integer, Integer> entry : lotsInfo.entrySet()) {  
            int num = entry.getKey();  
            int width = entry.getValue();  
            // check pre-condition, fail fast  
            if (width <= 0)  
                throw new Exception();  
            // work  
            Lot lot = new Lot(num, width);  
            this.lots.add(lot);  
        }  
        // check post-condition  
        assert this.getNumberOfLots() == lotsInfo.size();  
        assert this.status.size() == 0;  
        // check RI  
        checkRep();  
    }  
}
```

Coverage Data Table:

Element	Class, %	Method, %	Line, %	Branch, %
exercise_3_6.parkingfield	80% (4/5)	79% (34/43)	75% (158/209)	61% (105/171)
ConcreteParkingField	100% (1/1)	89% (17/19)	80% (120/150)	68% (98/143)
Lot	100% (1/1)	87% (7/8)	63% (14/22)	25% (4/16)
lotComparator	0% (0/1)	0% (0/1)	0% (0/7)	0% (0/4)
ParkingField	100% (1/1)	50% (1/2)	50% (1/2)	100% (0/0)
Record	100% (1/1)	69% (9/13)	79% (23/29)	50% (3/6)

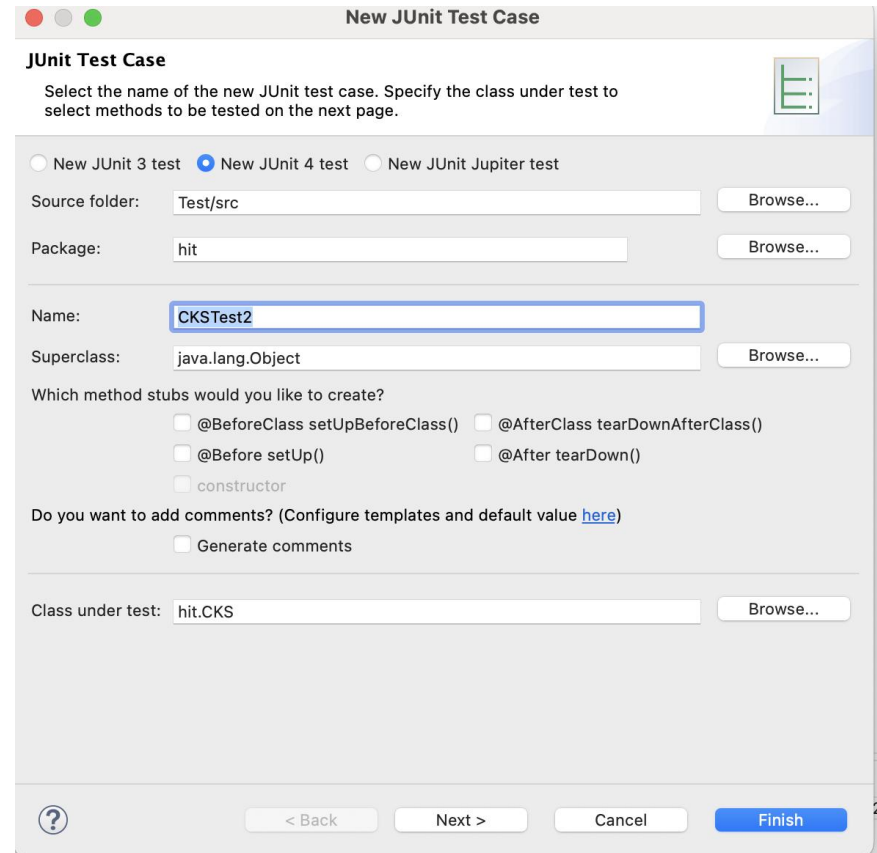
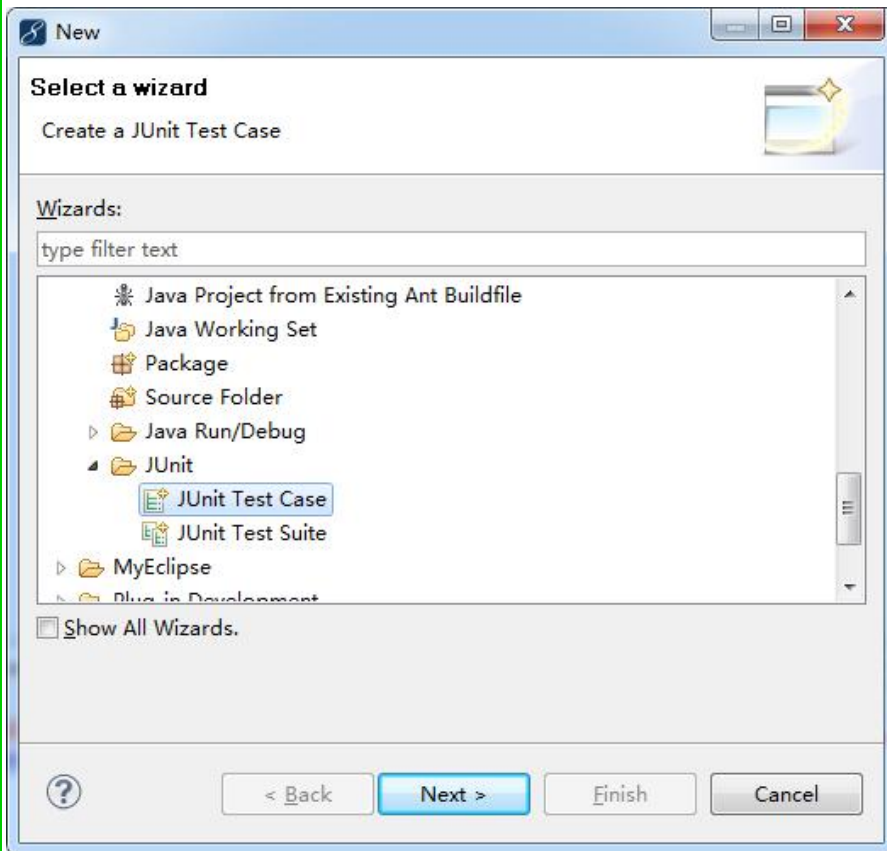
JUnit的安装与配置

- <https://junit.org/junit4/> <https://junit.org/junit5/>
- Eclipse和IDEA当前版本已经集成了JUnit。

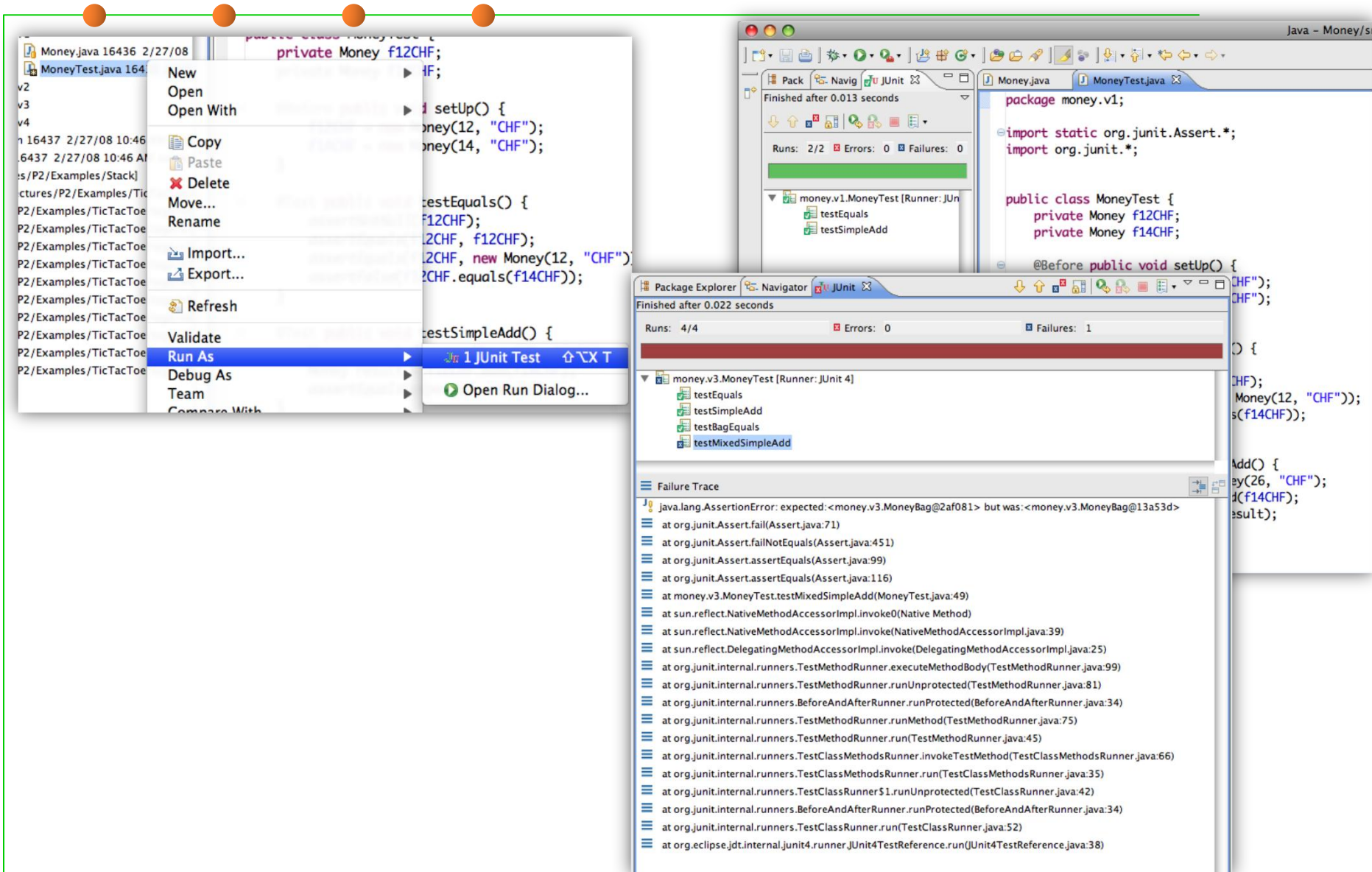


JUnit的安装与配置 Eclipse

- 针对一个存在的项目，在其中新建JUnit测试用例或测试套件；



在Eclipse中运行JUnit测试用例



JUnit的安装与配置 IDEA

- IDEA下使用JUnit <https://junit.org/junit4/>
 - 方法1: 下载 junit.jar 和 hamcrest-core.jar , 加入项目的Libraries中
 - 方法2: 如果使用Maven, 见网站说明

The screenshot shows the JUnit 4 website with the following content:

Download and Install
Marc Philipp edited this page on Sep 29, 2023 - 35 revisions

To download and install JUnit you currently have the following options.

Plain-old JAR

Download the following JARs and add them to your test classpath:

- [junit.jar](#)
- [hamcrest-core.jar](#)

Maven

Add a dependency to `junit:junit` in `test` scope. (Note: 4.13 is the latest stable version as of the latest edit on this page.)

```
<dependency>
<groupId>junit</groupId>
<artifactId>junit</artifactId>
<version>4.13</version>
<scope>test</scope>
</dependency>
```

Gradle

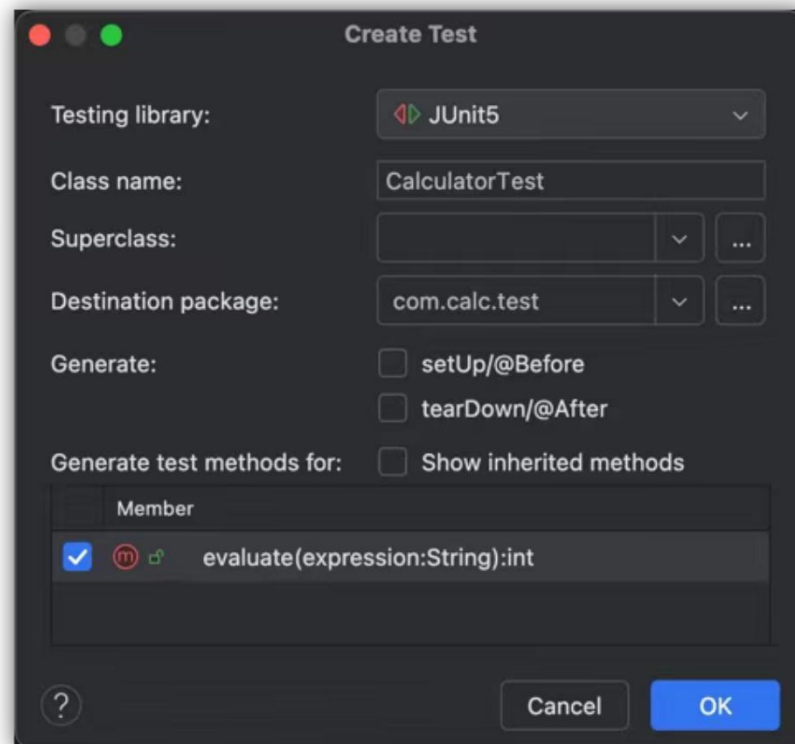
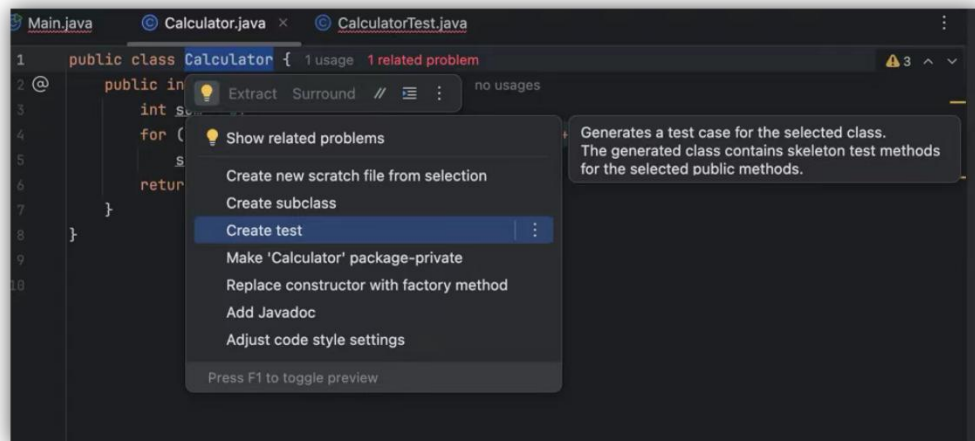
See [Use-with-Gradle](#)

On the right side, there is a sidebar with a search bar and a list of pages:

- Pages (35)
- Find a page...
- Home
- 'Enclosed' test runner example
- Aggregating tests in suites
- Assertions
- Assertions CN
- Assumptions with assume
- Categories
- Continuous testing
- Custom runners
- Deprecation Policy
- Developing with fast tests
- Download and Install
 - Plain-old JAR
 - Maven
 - Gradle
- Exception testing

JUnit的安装与配置 IDEA

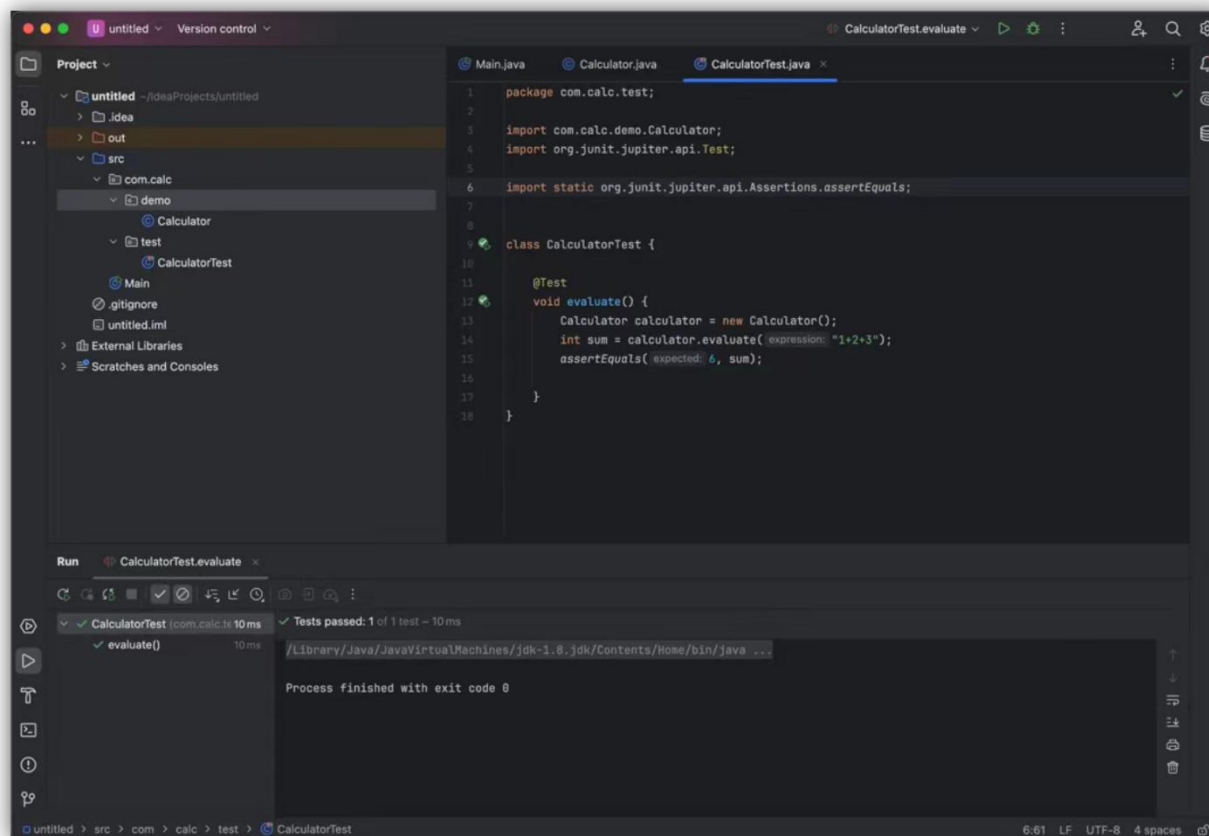
- IDEA下使用JUnit <https://junit.org/junit4/>
 - 方法3: 在需要测试的类或接口名称上使用 Ctrl+Shift+t, 然后选择创建测试。



JUnit的安装与配置 IDEA

■ IDEA下运行JUnit

- 在测试方法点击 IDEA 运行图标（或用 Ctrl+Shift+F10）运行单元测试。



JUnit4测试用例：小例子

```
public class Calculator {  
    public int evaluate(String expression) {  
        int sum = 0;  
        for (String summand: expression.split("\\+"))  
            sum += Integer.valueOf(summand);  
        return sum;  
    }  
}
```

```
import static org.junit.Assert.assertEquals;  
import org.junit.Test;
```

```
public class CalculatorTest {  
    @Test  
    public void evaluatesExpression() {  
        Calculator calculator = new Calculator();  
        int sum = calculator.evaluate("1+2+3");  
        assertEquals(6, sum);  
    }  
}
```

单元测试部分：实验评判标准

- 是否可正确配置JUnit/EclEmma;
- 所设计的测试用例是否满足要求;
- 是否正确书写了JUnit测试代码并执行, 以获得测试结果;
- 所涉及的测试用例的覆盖度;
- 使用Git情况。

提交方式

- 请遵循实验报告模板撰写。
- 提交日期：第14周周日晚(6月1日 23:55)
- 提交两个个文件到头歌平台：
 - 实验报告：命名规则“学号-Lab3-report.doc”，具体请参见模板
 - 增加了测试用例的源代码打包：命名规则“学号-Lab3-code.zip”
 - 同组的两人要分别提交
 - 确保GitHub上有本次实验之后的全部代码，包含所有测试用例
- 第13周和第14周的实验课上，请实验教师/TA现场检查测试执行情况。



結束



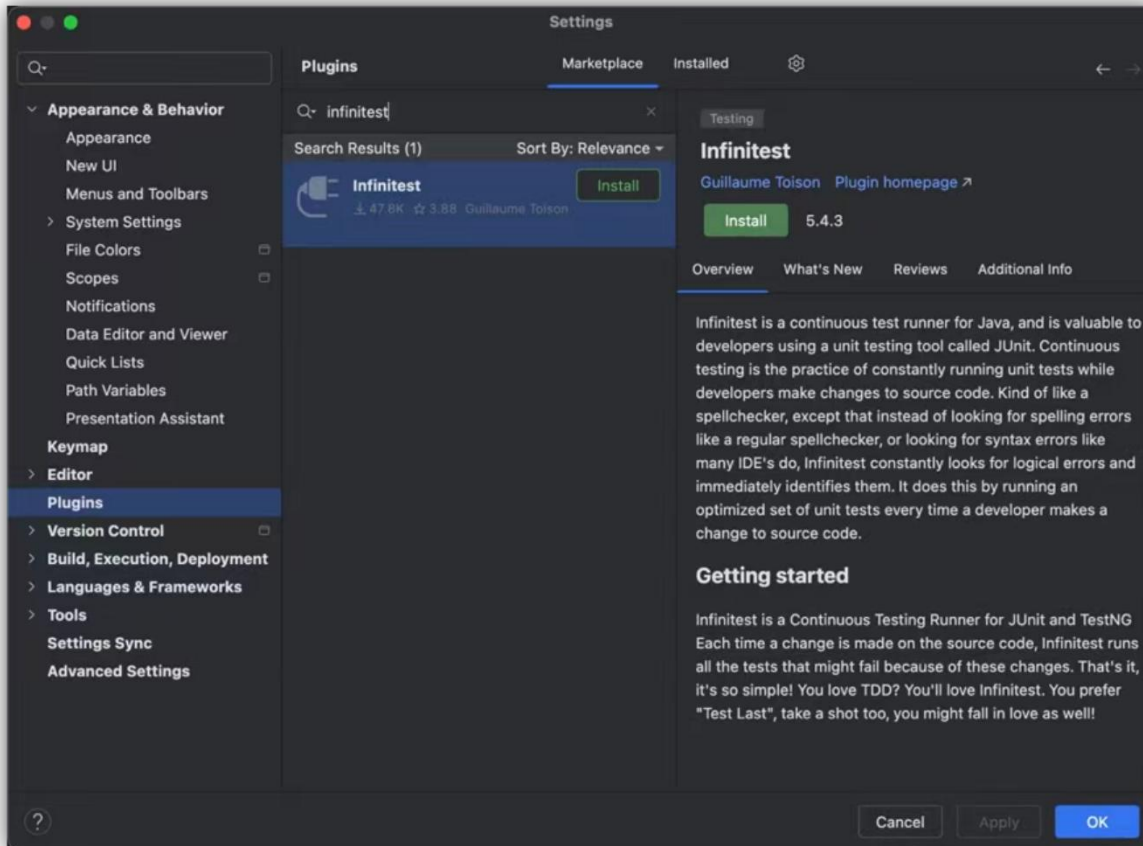
Infinitest 持续测试



- **Infinitest:** 针对Eclipse的持续测试插件 (Continuous Testing plugin);
- 每当源代码发生变化后, 所有受影响的测试用例都会被自动重新执行。
- <https://infinitest.github.io>

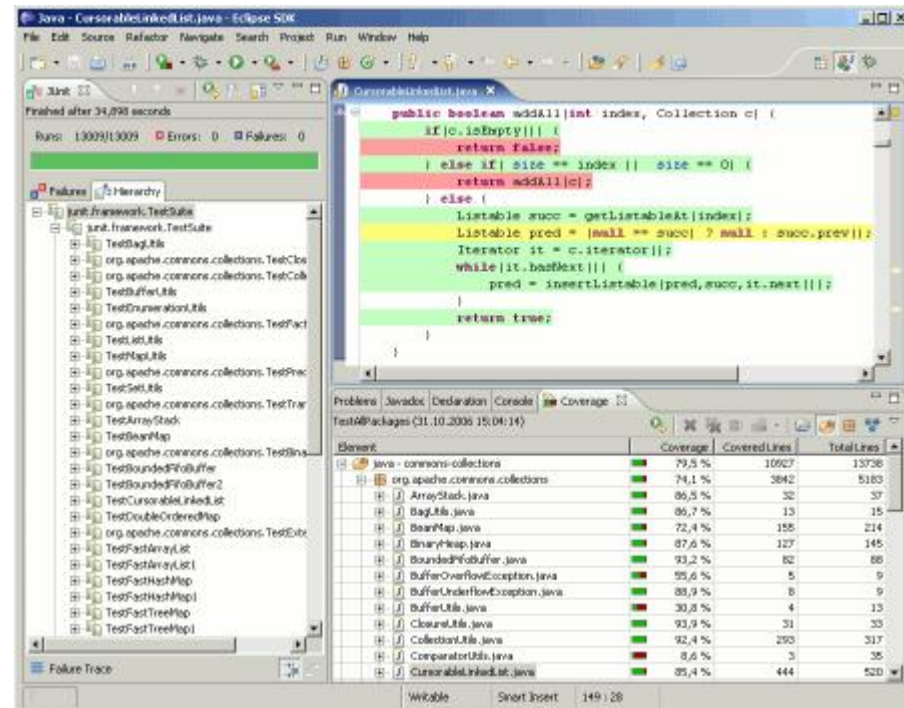
在idea中安装infinittest

Settings -> Plugins -> Browse repositories -> 查找
infinittest -> Install -> Restart idea



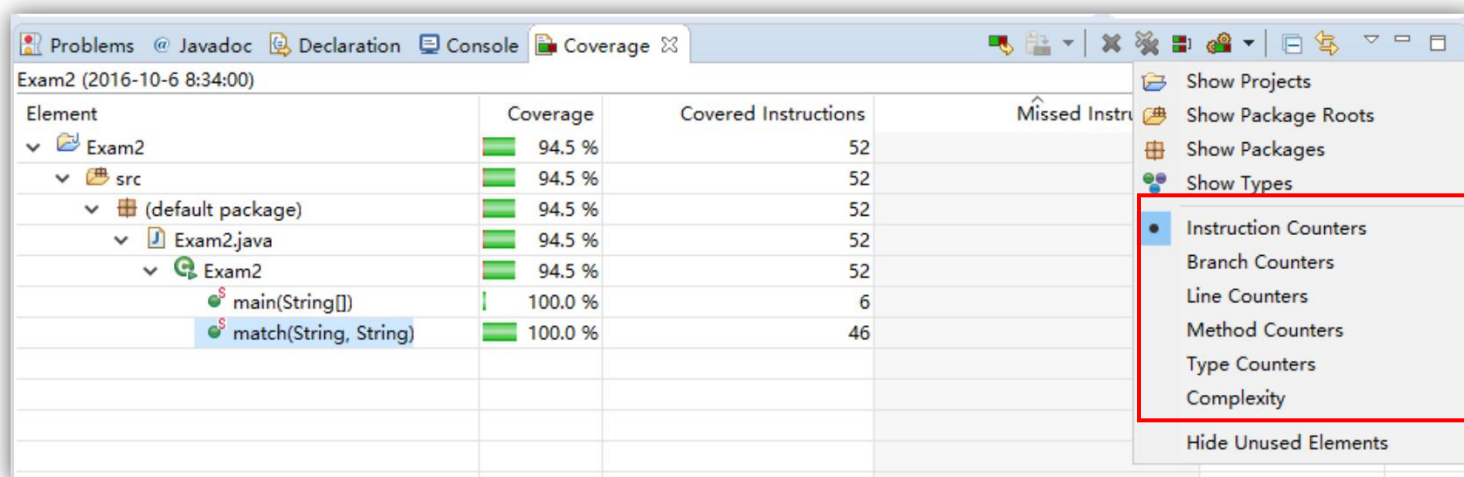
EclEmma

- 从<http://www.eclemma.org/download.html> 下载并配置到Eclipse中;
- 帮助教程:
 - <http://eclemma.org/userdoc/index.html>
 - <http://www.ibm.com/developerworks/cn/java/j-lo-eclemma>
- 针对前面你所设计的jUnit测试用例, 使用EclEmma分析它们的代码覆盖度, 并据此进一步完善测试用例, 确保覆盖度尽可能高。



EclEmma

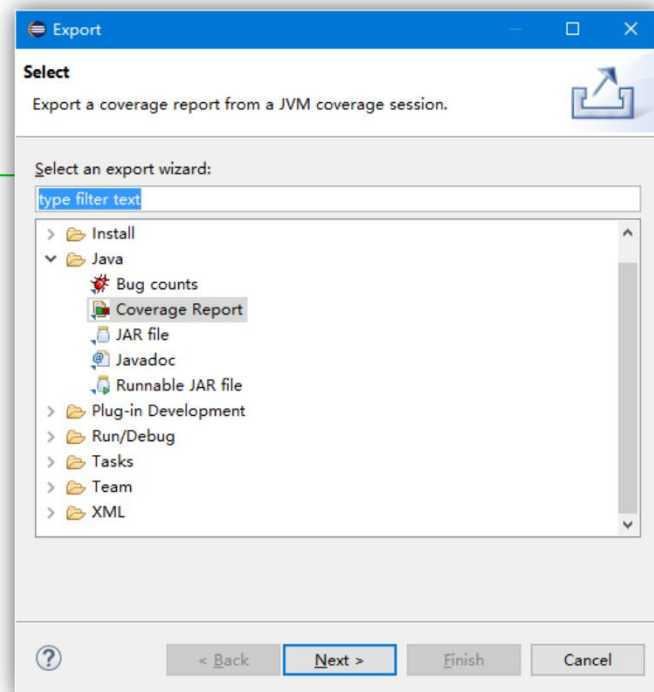
- 在Coverage视图的下拉菜单中选择不同的“覆盖度标准”
 - Instruction counters: 语句覆盖
 - Branch counters: 判定覆盖
 - Complexity: 基本路径覆盖



EclEmma

■ 导出覆盖度分析报告:

- 在Coverage视图的特定Element上右键菜单上选择“export session”，打开以下对话框，选择“Coverage Report”；



JaCoCo > org.jacoco.examples > org.jacoco.examples

org.jacoco.examples

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed Cxty	Missed Lines	Missed Methods	Missed Classes
ReportGenerator		0%		0%	7 7	28 28	6 6	1 1
ExecutionDataServer.Handler		0%		0%	5 5	26 26	4 4	1 1
MBearClient		0%		n/a	2 2	14 14	2 2	1 1
ExecutionDataClient		0%		n/a	2 2	14 14	2 2	1 1
ExecutionDataServer		0%		n/a	2 2	7 7	2 2	1 1
CoreTutorial.TestTarget		0%		0%	5 5	7 7	3 3	1 1
ClassInfo		95%		100%	1 5	2 17	1 4	0 1
ExecDump		93%		100%	1 8	2 23	1 5	0 1
CoreTutorial		97%		100%	1 11	2 44	1 6	0 1
ExecDump.new IExecutionDataVisitor() {...}		100%		n/a	0 2	0 3	0 2	0 1
CoreTutorial.MemoryClassLoader		100%		100%	0 4	0 8	0 3	0 1
ExecDump.new ISessionInfoVisitor() {...}		100%		n/a	0 2	0 3	0 2	0 1
Total	430 of 993	56%	8 of 26	69%	26 55	102 194	22 41	6 12

Code Coverage Report for JaCoCo 0.7.8-SNAPSHOT